

scripts README

Scripts README

This README explains how to set up the environment and run all scripts in this repository, from initial setup through to generating figures and evaluation results.

Project structure

```
scripts/
  additional_functions.R      # Shared plotting and utility functions
  nldr_code.R                # Wrapper functions for fitting NLDR methods
  evaluation.py              # Python evaluation metrics (global score, KNN, SVM, triplet)
  function_scripts/          # Low-level hexbin model fitting functions (one per NLDR method)
    Fit_PacMAP_code.py
    Fit_TriMAP_code.py
    functions_tri_hex_*.R
  five_gau_clusters/         # Five Gaussian clusters experiment
  c_shaped_dens_str/         # C-shaped density structure experiment
  two_nonlinear/             # Two nonlinear clusters experiment
  mnist/                    # MNIST digit-1 experiment
  pbmc3k/                   # PBMC3k single-cell RNA-seq experiment
```

1. One-time environment setup

1.1 R packages

Install the required R packages from CRAN and Bioconductor. Run once in R:

```
install.packages(c(
  "dplyr", "tibble", "readr", "ggplot2", "GGally",
  "conflicted", "here", "Rtsne", "umap", "phateR",
  "reticulate", "hexbin", "patchwork", "ggbeeswarm",
  "Seurat"      # pbmc3k only
))
```

1.2 Python environment (conda)

The R scripts use `reticulate` to call Python for PaCMap and TriMAP. A dedicated conda environment is required.

Create and activate the environment:

```
conda create -n pcamp_env python=3.9
conda activate pcamp_env
```

Install required Python packages:

```
pip install pacmap trimap scikit-learn numpy numba matplotlib
```

Verify the environment path — the scripts expect Python at:

```
~/miniforge3/envs/pcamp_env/bin/python
```

If your miniforge/miniconda is installed elsewhere, update the path in any script that contains:

```
use_python("~/miniforge3/envs/pcamp_env/bin/python")
use_condaenv("pcamp_env")
```

1.3 Verify reticulate connection

Open R and run:

```
library(reticulate)
use_condaenv("pcamp_env")
py_config()           # should show the pcamp_env Python path
reticulate::import("pacmap") # should import without error
reticulate::import("trimap") # should import without error
```

2. Shared files

These files are sourced or imported inside experiment scripts and should not be run directly.

File	Language	Purpose
additional_functions.R	R	Plotting helpers: <code>plot_proj()</code> for grand tour views, <code>plot_digit_img()</code> for MNIST images, <code>plot_hbe()</code> / <code>plot_hbe_nbar()</code> for error curves, <code>scale_data_manual()</code> for data scaling, <code>interior_annotation()</code> for figure labels
nldr_code.R	R	Wrapper functions for fitting NLDL methods: <code>Fit_tsNE()</code> , <code>Fit_UMAP()</code> , <code>Fit_PHATE()</code> , <code>Fit_TrimAP_data()</code> , <code>Fit_PacMAP_data()</code> , plus corresponding 2D plot functions

File	Language	Purpose
evaluation.py	Python	Evaluation metrics: global_score(), knn_eval(), faster_svm_eval(), random_triplet_eval(), centroid_triplet_eval(). Sourced via <code>reticulate</code> in evaluation scripts
function_scripts/Fit_PaCMAP.R	R	Low-level PaCMAP fitting called via <code>reticulate::source_python()</code>
function_scripts/Fit_TriMAP.R	R	Low-level TriMAP fitting called via <code>reticulate::source_python()</code>
function_scripts/functions_tri_hex_*.R	R	Hexbin model fitting functions, one file per NLDR method and variant (with/without PCA pre-processing). Sourced inside experiment scripts

3. Running the experiments

All scripts use `here::here()` for file paths, so they must be run from the **project root directory** (the folder containing `paper-nldr-vis-algorithm.Rproj`). Open the `.Rproj` file in RStudio, or set your working directory explicitly:

```
setwd("/path/to/paper-nldr-vis-algorithm")
```

Scripts within each subfolder are numbered and must be run **in order**.

3.1 Five Gaussian clusters (`five_gau_clusters/`)

Order	Script	What it does
1	01_five_gaussian_cluster_data_emb.R	Generates the 5-cluster dataset and generates all NLDR embeddings (t-SNE, UMAP, PaCMAP, PHATE, TriMAP)
2	02_gen_model_with_tSNE.R	Fits the hexbin lifted model on the t-SNE embedding; computes goodness-of-fit errors and generates grand tour projection objects
3	03_gen_model_with_UMAP.R	Same for UMAP
4	04_gen_model_with_PaCMAP.R	Same for PaCMAP

Python required: Yes — 01_ uses `reticulate` for PaCMAP and TriMAP.

```
Rscript scripts/five_gau_clusters/01_five_gaussian_cluster_data_emb.R
Rscript scripts/five_gau_clusters/02_gen_model_with_tSNE.R
Rscript scripts/five_gau_clusters/03_gen_model_with_UMAP.R
Rscript scripts/five_gau_clusters/04_gen_model_with_PaCMAP.R
```

3.2 C-shaped density structure (c_shaped_dens_str/)

Order	Script	What it does
1	01_gen_data.R	Simulates both the non-uniform and uniform density C-shaped datasets
2	02_gen_embeddings_uni_dens.R	Generates NLDR embeddings for both datasets across multiple hyperparameter settings
3	03_gen_model_with_tSNE.R	Fits the hexbin lifted model on the t-SNE embedding; computes errors and generates grand tour objects including selected/deselected point classifications

Python required: Yes — 02_ uses `reticulate` for PaCMAP and TriMAP.

```
Rscript scripts/c_shaped_dens_str/01_gen_data.R
Rscript scripts/c_shaped_dens_str/02_gen_embeddings_uni_dens.R
Rscript scripts/c_shaped_dens_str/03_gen_model_with_tSNE.R
```

3.3 Two nonlinear clusters (two_nonlinear/)

Order	Script	What it does
1	01_gen_data.R	Simulates the two-cluster dataset and creates train/test splits
2	02_gen_true_model.R	Generates the analytically defined true model manifold and its edge connections
3	03_gen_embeddings.R	Generates all NLDR embeddings (t-SNE, UMAP, PaCMAP, PHATE, TriMAP) including UMAP predictions for the test set and true model
4	04_gen_mse_for_diff_methods.R	Computes hexbin-level goodness-of-fit error summaries across methods and grid sizes
5	05_gen_rm_lwd_mse.R	Runs error sensitivity analysis varying the low-weight-drop threshold
6	06_gen_model_with_tSNE.R	Fits the hexbin lifted model on t-SNE; generates grand tour projection objects and transition flow data
7	07_example_evaluation_metrics.R	Computes standard NLDR evaluation metrics (R_NX curves, Shepard diagram data)
8	08_gen_model_with_PHATE.R	Fits the hexbin lifted model on PHATE; generates grand tour projection objects
9	09_gen_model_prediction_with_umpred_and_tripred.R	Compares UMAP and TriMAP approach prediction against the hexbin model prediction; computes pairwise distance comparisons across bin resolutions
—	compare_pred_approaches.R	Standalone comparison script (does not depend on order; can be run after script 9)

Python required: Yes — 03_ uses `reticulate` for PaCMAP and TriMAP.

```

Rscript scripts/two_nonlinear/01_gen_data.R
Rscript scripts/two_nonlinear/02_gen_true_model.R
Rscript scripts/two_nonlinear/03_gen_embeddings.R
Rscript scripts/two_nonlinear/04_gen_mse_for_diff_methods.R
Rscript scripts/two_nonlinear/05_gen_rm_lwd_mse.R
Rscript scripts/two_nonlinear/06_gen_model_with_tSNE.R
Rscript scripts/two_nonlinear/07_example_evaluation_metrics.R
Rscript scripts/two_nonlinear/08_gen_model_with_PHATE.R
Rscript scripts/two_nonlinear/09_gen_model_prediction_with_umap_and_our_approach.R
Rscript scripts/two_nonlinear/compare_pred_approaches.R

```

3.4 MNIST (mnist/)

Order	Script	What it does
1	01_data_preprocessing.R	Loads the raw MNIST data, subsets digit-1 images, and computes the top 10 PCs
2	02_gen_diff_embeddings.R	Generates all NLDR embeddings across multiple hyperparameter settings (t-SNE perplexity 30/40/89, UMAP, PaCMAP, PHATE, TriMAP)
3	03_gen_mse_for_diff_methods.R	Computes hexbin-level goodness-of-fit error summaries for all methods
4	04_gen_model_with_tSNE.R	Fits the hexbin lifted model on the t-SNE embedding; computes per-point errors and generates grand tour projection objects; samples digit images from error regions
5	05_evaluation_metrics.R	Computes standard NLDR evaluation metrics (R _{NX} curves, Shepard diagram, summary table)
6	06_link_brush_layout_e.R	Generates the linked brushing layout figure showing error regions alongside digit images

Python required: Yes — 02_ uses reticulate for PaCMAP and TriMAP.

```

Rscript scripts/mnist/01_data_preprocessing.R
Rscript scripts/mnist/02_gen_diff_embeddings.R
Rscript scripts/mnist/03_gen_mse_for_diff_methods.R
Rscript scripts/mnist/04_gen_model_with_tSNE.R
Rscript scripts/mnist/05_evaluation_metrics.R
Rscript scripts/mnist/06_link_brush_layout_e.R

```

3.5 PBMC3k (pbmc3k/)

This is the most involved experiment. It includes a large hyperparameter sweep and a comparison with the scDEED method.

Order	Script	What it does
1	01_obtain_pca_author.R	Processes the raw Seurat PBMC3k object and extracts the top 50 PCs
2	02_obtain_umap_authors.R	Reproduces the authors' original UMAP embedding
3	03_gen_umap_diff_param.R	Generates UMAP embeddings across the full $n_neighbors \times min_dist$ hyperparameter sweep
4	04_gen_tsne_diff_param.R	Generates t-SNE embeddings across multiple perplexity values
5	05_gen_phate.R	Generates PHATE embeddings for $k = 2, 5, 10$
6	06_gen_trimap.R	Generates TriMAP embeddings for three hyperparameter configurations
7	07_gen_pacmap.R	Generates PaCMAP embeddings for four hyperparameter configurations
8	08_gen_mse_for_diff_methods.R	Computes hexbin-level goodness-of-fit error summaries for all methods
9	09_gen_scDEED.R	Runs the scDEED evaluation on t-SNE and UMAP hyperparameter sweeps using the 5858-cell dataset
10	10_pre_process_for_embedding.R	Prepares the 5858-cell scDEED dataset for embedding
11	11_gen_mse_for_diff_tsne_scD.R	Computes goodness-of-fit error summaries for t-SNE configurations on the scDEED dataset
12	12_gen_mse_for_diff_umap_scD.R	Same for UMAP configurations on the scDEED dataset
13	13_gen_model_with_UMAP.R	Fits the hexbin lifted model on the primary UMAP embedding; generates grand tour projection objects
14	14_gen_model_with_tSNE.R	Same for the primary t-SNE embedding
15	15_gen_model_with_UMAP_scD.R	Fits the hexbin lifted model on the UMAP scDEED embedding
16	16_gen_model_with_tSNE_scD.R	Same for the t-SNE scDEED embedding
17	17_evaluation_metrics.R	Computes standard NLDR evaluation metrics for the primary 2622-cell embeddings
18	18_evaluation_metrics_scD.R	Same for the scDEED 5858-cell embeddings

Python required: Yes — scripts 3, 6, 7 use `reticulate` for PaCMAP and TriMAP.

Additional R package required: Seurat (for scripts 1–2).

```
Rscript scripts/pbmc3k/01_obtain_pca_author.R
Rscript scripts/pbmc3k/02_obtain_umap_authors.R
Rscript scripts/pbmc3k/03_gen_umap_diff_param.R
Rscript scripts/pbmc3k/04_gen_tsne_diff_param.R
Rscript scripts/pbmc3k/05_gen_phate.R
Rscript scripts/pbmc3k/06_gen_trimap.R
Rscript scripts/pbmc3k/07_gen_pacmap.R
Rscript scripts/pbmc3k/08_gen_mse_for_diff_methods.R
Rscript scripts/pbmc3k/09_gen_scDEED.R
Rscript scripts/pbmc3k/10_pre_process_for_embedding.R
Rscript scripts/pbmc3k/11_gen_mse_for_diff_tsne_scD.R
Rscript scripts/pbmc3k/12_gen_mse_for_diff_umap_scD.R
```

```
Rscript scripts/pbmc3k/13_gen_model_with_UMAP.R
Rscript scripts/pbmc3k/14_gen_model_with_tSNE.R
Rscript scripts/pbmc3k/15_gen_model_with_UMAP_scD.R
Rscript scripts/pbmc3k/16_gen_model_with_tSNE_scD.R
Rscript scripts/pbmc3k/17_evaluation_metrics.R
Rscript scripts/pbmc3k/18_evaluation_metrics_scD.R
```

4. Recommended run order across experiments

If reproducing everything from scratch, run experiments in this order (each is independent but later experiments assume the shared setup is working):

1. `five_gau_clusters/` — simplest, good for verifying the full pipeline works
2. `c_shaped_dens_str/`
3. `two_nonlinear/`
4. `mnist/`
5. `pbmc3k/`

5. Notes and common issues

Seed: All experiments set `set.seed(20240110)` at the top of the embedding scripts. Results will be exactly reproducible only when using the same R and Python package versions.

Python path: If `use_python()` or `use_condaenv()` fails, check the output of `conda env list` in your terminal and update the path in the relevant script.

umap vs uwot: The scripts use the `umap` package (not `uwot`) because `uwot`'s `predict()` method was not available at the time of writing. Do not substitute `uwot`.

here::here(): All file paths are relative to the project root via `here::here()`. Always open the `.Rproj` file or set your working directory to the project root before running any script.